

뉴스 인텔리전스 파이프라인

검색어 하나로 시작하는
기사 수집 · 자동 요약 · 감정 분석

PRESENTED BY

👤 [발표자 이름]

DATE

📅 2026. 01. 04

WEB DEMO miniproject02.vercel.app

STREAMLIT miniproject02...streamlit.app



뉴스 과잉 속 핵심만 빠르게 이해하는 기술

검색부터 요약, 감정 분석까지

한 번에 처리하는

표준화된 뉴스 파이프라인



중단 없는 서비스 안정성

3단계 폴백 시스템 [OpenAI → Local → Stub](#) 탑재로 API 키나 모델 부재 시에도 항상 결과를 반환합니다.



표준 JSON 스키마

모든 데이터가 통일된 구조로 출력되어 저장, 불러오기, UI 확장 등 다양한 환경에서 즉시 재사용 가능합니다.



멀티 인터페이스 지원

하나의 코어로 **Streamlit** 통합 웹, **React** 분리형 웹, **Tkinter** 데스크톱 앱까지 모두 지원합니다.

정보의 홍수와 비효율적인 분석 과정

"매일 쏟아지는 수천 건의 뉴스,
핵심을 파악하는 데
너무 많은 시간이 걸립니다."

현대 지식 근로자는 정보 과잉 시대에 살고 있습니다. 단순히 기사를 검색하는 것을 넘어, 내용을 읽고 요약하며 긍정/부정 여부까지 판단하는 반복적인 과정은 막대한 시간과 인지적 비용을 소모하게 만듭니다.

또한 웹, 모바일, 데스크톱 등 다양한 환경에서 일관되지 않은 데이터 포맷은 자동화와 재사용을 방해하는 가장 큰 걸림돌입니다.



압도적인 정보량 (Volume)

수백 개의 검색 결과 중에서 진짜 중요한 뉴스를 선별하기 어렵고, 모든 기사를 정독하는 것은 물리적으로 불가능합니다.



반복적인 수작업 (Process)

검색 → 독해 → 요약 → 감정 파악의 4단계 프로세스를 사람이 직접 수행하면 속도가 느리고 비용이 급증합니다.



파편화된 포맷 (Consistency)

서비스 환경별로 출력 형식이 제각각이라, 한 번 구축한 로직을 다른 플랫폼(웹, 앱, 로컬)에서 재사용하기 어렵습니다.

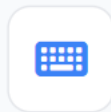
SOLUTION OVERVIEW

표준 파이프라인 중심 설계 One Source, Multi-Use

"검색어 입력 한 번으로
수집부터 분석까지
완벽하게 처리합니다."

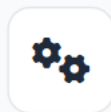
이 프로젝트의 핵심 솔루션은 어떤 환경에서도 동일하게 동작하는 강력한 데이터 파이프라인입니다. 복잡한 로직은 백엔드에 숨기고, 사용자는 단순한 입력만으로 고도로 정제된 정보를 얻을 수 있습니다.

특히 입출력 규격을 **표준 JSON 스키마**로 고정하여, 웹(Streamlit/React)부터 데스크톱(Tkinter)까지 모든 인터페이스에서 로직을 100% 재사용합니다.



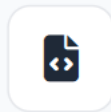
1. 정형화된 입력 (Input)

`query`, `language`, `sort_by` 등 필수 파라미터만으로 뉴스 수집 요청을 단순화했습니다.



2. 지능형 처리 (Process)

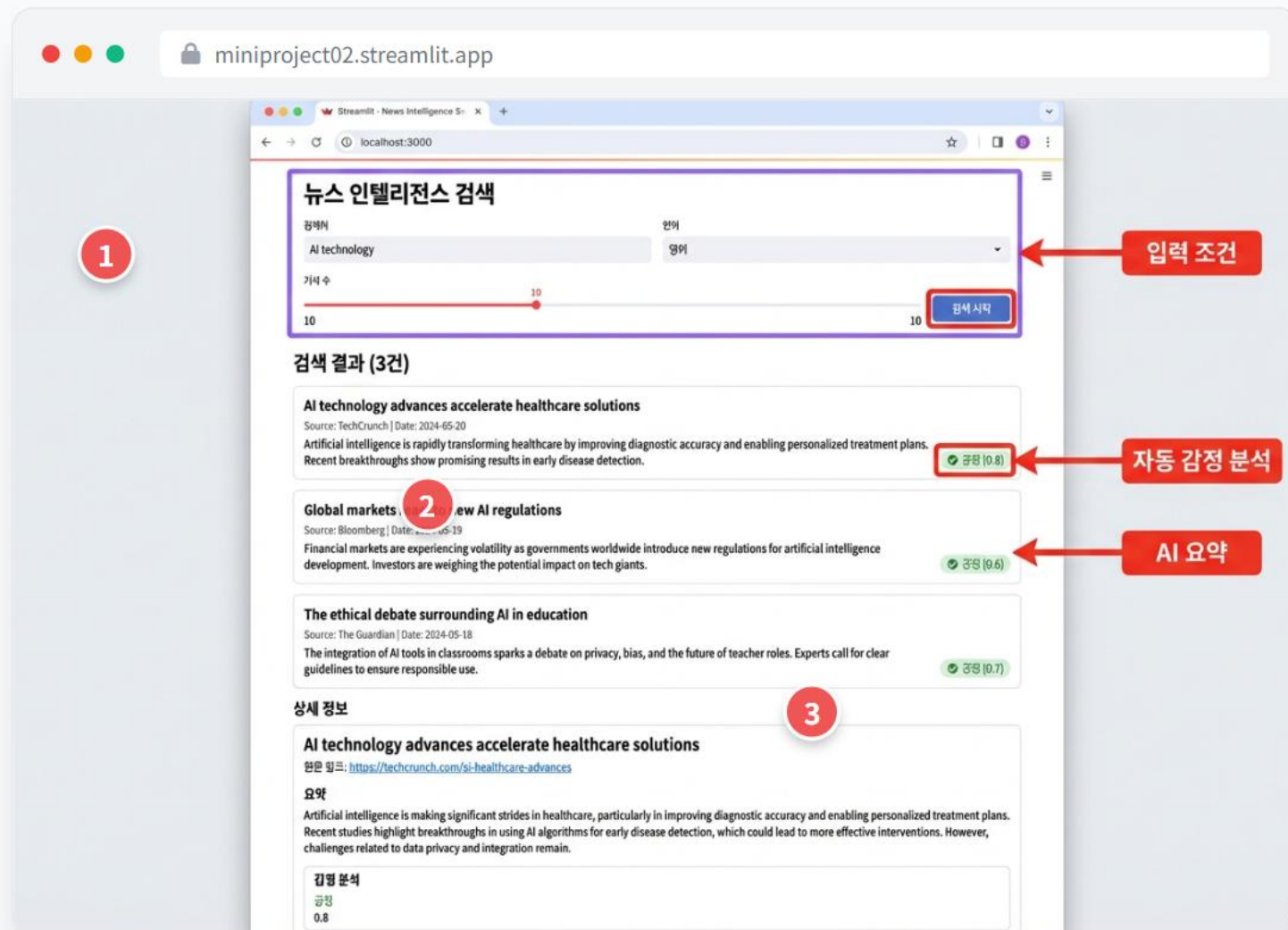
NewsAPI 수집 → 텍스트 정규화 →
AI 요약 및 감정 분석(3단계 폴백) 자동 수행



3. 표준 JSON 출력 (Output)

기사 원문, 요약문, 감정 스코어(`label`, `score`)가 포함된 단일 JSON 객체를 반환합니다.

Streamlit 통합 앱: 한 화면에서 모든 분석 완료



1

통합 검색 제어

검색어, 언어, 기사 수집 개수(Page Size) 등 모든 파라미터를 사이드바에서 직관적으로 설정합니다.

2

실시간 목록 & 요약

수집된 기사의 핵심 내용을 AI가 즉시 요약하고, 긍정/부정 감정 상태를 배지 형태로 시각화합니다.

3

상세 분석 뷰 (Detail View)

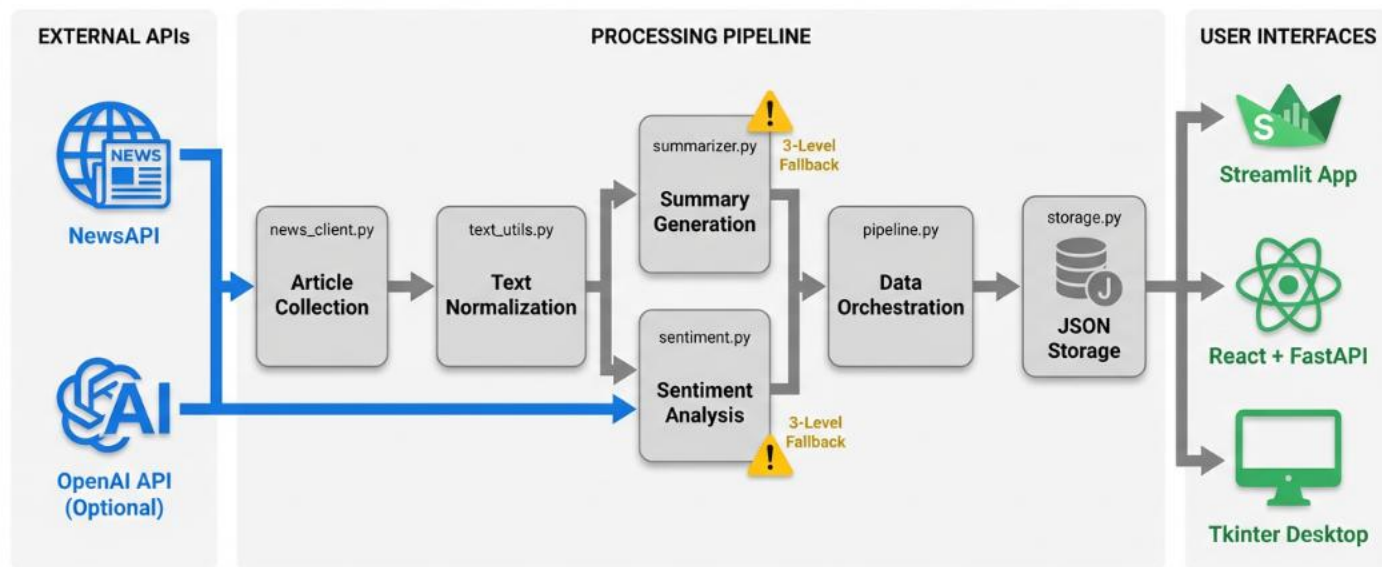
원문 링크, 텍스트 전문, 모델별 분석 상세 정보를 탭이나 확장 패널로 깊이 있게 탐색할 수 있습니다.



데이터 자산화 (JSON)

분석 결과를 표준 JSON 포맷으로 즉시 저장하거나, 이전에 저장된 파일을 불러와 결과를 재현합니다.

아키텍처 & 데이터 플로우



Step 1 수집 및 정규화

news_client.py가 외부 API를 호출하고, text_utils.py가 제목/본문을 병합하여 분석 가능한 텍스트로 정제합니다.



Step 2 AI 분석 코어 (Fallback)

summarizer.py와 sentiment.py가 OpenAI → Local Model → Stub 순으로 폴백하며 요약과 감정 분석을 수행합니다.



Step 3 표준 출력 및 인터페이스

pipeline.py가 표준 JSON 스키마로 데이터를 조립하여 Streamlit, React, Desktop UI 등 모든 채널에 동일한 결과를 제공합니다.

핵심 기술 & 방법론

안정적인 파이프라인 구축을 위한 기술적 접근 방식



텍스트 정규화 (Normalization)

제목, 요약, 본문을 병합하고 불필요한 공백을 제거하여 모델이 이해하기 쉬운 형태로 가공합니다.

text_utils.py



하이브리드 요약 (Hybrid Summarization)

OpenAI(품질) → Transformers(로컬) → Stub(안정성) 순으로 자동 전환되는 폴백 체인을 구현했습니다.

summarizer.py



감정 분석 (Sentiment Analysis)

LLM의 문맥 이해 능력과 VADER의 룰 기반 속도를 결합하여, 비용 효율적이고 안정적인 분석을 제공합니다.

sentiment.py

pipeline_logic.py

```
01 class NewsPipeline:
02     def process_article(self, article):
03         # 1. 텍스트 정규화 및 병합
04         clean_text = text_utils.normalize(article.title,
05                                           article.text)
06
07         # 2. 요약 (3단계 폴백 자동 적용)
08         try:
09             summary = self.summarizer.summarize(clean_text)
10         except Exception:
11             summary = self.summarizer.fallback_stub(clean_text)
12
13         # 3. 감정 분석 (OpenAI → VADER)
14         sentiment = self.sentiment.analyze(clean_text)
15
16         return {
17             "summary": summary,
18             "sentiment": sentiment,
19             "meta": article.metadata
```

3단계 폴백 시스템 (3-Level Fallback)

API 키가 없어도, 모델이 없어도, 서비스는 멈추지 않습니다.

3단계 폴백 시스템



✓ 항상 결과 제공 | ✓ 서비스 중단 없음 | ✓ 환경 변화에 강함

Image 8

1

Cloud AI



✓ OpenAI GPT 모델 사용

요약 문맥을 이해한 생성형 요약

감정 뉘앙스까지 파악하는 분석

Highest Quality

2

Local Models



↔ 키/쿼터 오류 시 자동 전환

요약 HuggingFace Transformers

감정 VADER (Rule-based)

Performance Balanced

3

Stub Fallback



! 라이브러리 부재 시 최후 수단

요약 텍스트 앞부분 절단(Truncate)

감정 단순 키워드 매칭

Guaranteed Reliability

표준 JSON 스키마

Data Consistency & Reusability

output_schema.json

```
"query": "AI technology", "language": "en", "fetch_at":  
"2026-01-04T10:00:00Z", "articles": [ {  
  "id": "a1b2c3d4 ... ", // SHA256 Hash  
  "title": "The Future of AI ... ", "url":  
  "https://example.com/ ... ", "source": "TechCrunch",  
  "summary": "AI agents are evolving ... ", // AI 요약문  
  "sentiment": { "label": "positive", "score": 0.85, "model":  
  "openai-gpt-4o" }  
} ]
```



고유 식별자 (ID)

URL 또는 제목을 기반으로 **SHA256 해시**를 생성하여 기사를 고유하게 식별합니다. 중복 수집 방지와 캐싱의 키(Key)로 활용됩니다.

pipeline.py

id_generation



AI 분석 데이터 통합

단순 메타데이터뿐만 아니라, AI가 생성한 **요약문(Summary)**과 **감정 분석(Sentiment)** 결과를 하나의 객체 안에 캡슐화합니다.

Enriched Data

One-Pass



재사용성과 호환성

동일한 JSON 구조 덕분에 **Streamlit, React, Desktop** 등 어떤 UI에 서도 데이터 파싱 로직을 수정 없이 그대로 공유합니다.

Cross-Platform

Standardized

멀티 인터페이스 전략

하나의 코어 로직으로 다양한 운영 환경을 완벽하게 지원합니다



streamlit_app.py

Streamlit 통합 앱

ALL-IN-ONE

데이터 탐색과 프로토타이핑에 최적화된 통합 환경입니다. 별도의 프론트엔드 구축 없이 Python 코드만으로 대시보드를 완성합니다.

- ✓ 빠른 배포 (Community Cloud)
- ✓ 단일 스크립트 실행
- ✓ 직관적인 사이드바 설정



App.jsx / main.py

React + FastAPI

PRODUCTION READY

실제 서비스 운영을 위한 분리형 아키텍처입니다. 확장 가능한 REST API 백엔드와 현대적인 UX의 프론트엔드를 제공합니다.

- ✓ 프론트/백엔드 독립 배포
- ✓ 표준 REST API (/api/fetch)
- ✓ Vite 기반 최신 번들링



app.py

Tkinter 데스크톱

NATIVE DESKTOP

브라우저 없이 로컬 환경에서 가볍게 동작하는 네이티브 앱입니다. 인터넷 연결이 제한된 환경이나 개인 도구로 적합합니다.

- ✓ 경량화된 실행 (No Browser)
- ✓ 로컬 파일 시스템 직접 제어
- ✓ Python 표준 라이브러리 활용

Powered by Shared Core → **pipeline.py** → Unified Logic

배포 & 기술 스택

최신 Python 생태계와 모던 프레임워크를 결합한 유연한 아키텍처



Python 3.9+

Core Engine

INTELLIGENCE LAYER

데이터 수집부터 정제, 요약, 감정 분석까지 담당하는 파이프라인의 핵심 엔진입니다. 오픈소스 모델과 API를 효율적으로 결합합니다.

- ✓ Transformers (Hugging Face)
- ✓ VADER Sentiment Analysis
- ✓ NewsAPI Client Integration



Full Stack

App Frameworks

INTERACTIVE UX

다양한 사용자 요구사항을 충족하기 위해 웹과 데스크톱 환경을 모두 지원하는 모던 프레임워크를 사용합니다.

- ✓ Streamlit (Rapid Dashboard)
- ✓ FastAPI (High Perf. Backend)
- ✓ React + Vite (Modern Frontend)



DevOps

Deployment & Sec

CLOUD & SECURITY

서비스의 확장성과 보안을 고려한 배포 전략을 제공합니다. API 키 관리와 환경별 최적화된 호스팅을 지원합니다.

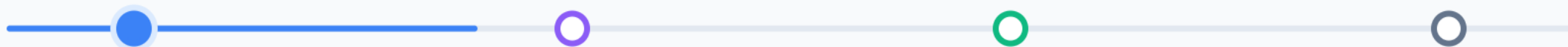
- ✓ Streamlit Cloud / Vercel / Render
- ✓ 환경변수 및 시크릿 관리
- ✓ CORS & Rate Limiting

Built with ❤️ &

Open Source Ecosystem

로드맵 & 넥스트 스텝

단기 품질 개선부터 장기적인 제품화 및 보안 강화 계획



PHASE 1

품질 고도화

- 스텝(Stub) 요약 제거
문서 구조를 고려한 추출+생성 하이브리드 모델 적용
- 뉴스 특화 감정 분석
금융/정치 도메인 사전 학습 모델 도입
- 스코어 캘리브레이션 (중립/불확실성 처리)

Immediate Focus

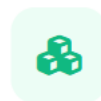


PHASE 2

신뢰성/관측성

- 에러 세분화
Rate limit, Timeout 등 원인을 UI에 명확히 노출
- 지능형 캐싱
동일 쿼리/정렬 조건에 대한 Redis 캐시 적용
- 폴백 사용률 및 지연 시간 모니터링

Q1 2026



PHASE 3

제품화 기능

- 사용자 경험 확장
즐거찾기, 키워드 알림, 검색어 히스토리
- 인사이트 가공
주제별 클러스터링 및 핵심 이슈 타임라인 시각화
- CSV/DB 저장 및 결과 리포트 다운로드

Q2 2026



PHASE 4

배포 및 보안

- 시크릿 관리 강화
환경변수 외 전용 Secret Manager 도입
- API 하드닝
CORS 정책 강화, IP 기반 Rate Limiting 적용
- 프론트엔드 API 키 노출 원천 차단 (Proxy)

Long Term

함께 성장할 파트너를 찾습니다

뉴스 인텔리전스 파이프라인의
고도화와 실제 비즈니스 적용을 위한
구체적인 협력 제안



베타 테스터 및 파트너 모집

초기 프로토타입을 직접 사용해보고 실질적인 UI/UX 피드백을 제공해 주실 얼리어답터 및 디자인 파트너를 환영합니다.



운영 비용 및 리소스 지원

안정적인 NewsAPI 유료 플랜 구독과 고성능 LLM 모델 활용을 지속하기 위한 서버/API 비용 투자가 필요합니다.



데이터 확장 및 엔터프라이즈 PoC

뉴스 외 사내 문서나 산업 리포트 등 데이터 소스를 연동하고, 기업 맞춤형 솔루션을 검증할 PoC 기회를 제안합니다.

부록: 코드 스니펫 — summarizer.py

OpenAI → Transformers → Stub 순서로 이어지는 3단계 요약 폴백 구현



PRIORITY 1

OpenAI API 요약

API 키가 존재할 경우 가장 먼저 시도합니다. 문맥을 이해하는 고품질 요약을 제공합니다.



PRIORITY 2

Local Transformers

OpenAI 호출 실패 시 로컬 모델(BART/T5 등)로 자동 전환됩니다. 외부 네트워크 의존 없이 작동합니다.



FALLBACK

Stub (안전 장치)

모든 AI 모델 실패 시 텍스트 앞부분을 잘라서 반환합니다. 서비스가 멈추지 않게 보장합니다.

summarizer.py

```
01 class Summarizer:
02     def summarize(self, text):
03         # 1. Try OpenAI (Highest Quality)
04         if self.openai_client:
05             try: return self._summarize_openai(text)
06             except Exception as e:
07                 print(f"OpenAI failed: {e}")
08
09         # 2. Try Local Transformers (Offline)
10         if self.use_transformers:
11             try: return self._summarize_local(text)
12             except Exception as e:
13                 print(f"Local model failed: {e}")
14
15         # 3. Final Fallback: Stub (Reliability)
16         return self._summarize_stub(text)
17
18     def _summarize_stub(self, text):
19         return text[:200] + "..."
```

실행 가이드 (Execution Guide)

로컬 환경 설정 및 인터페이스별 실행 방법

⚙️ 환경 설정 (Setup)

1. 필수 패키지 설치

Terminal

```
bash

$ git clone https://github.com/...
$ pip install -r requirements.txt
# 모든 의존성 라이브러리(FastAPI, Streamlit 등) 일괄 설치
```

2. 환경 변수 설정 (.env)

Config

```
.env

NEWS_API_KEY= "a1b2 ..."
OPENAI_API_KEY= "sk- ..."
USE_VADER= 1
# 키가 없으면 스템(Stub) 모드로 자동 동작
```

필수

선택

▶ 인터페이스 실행 (Run)

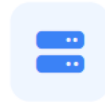


Streamlit 통합 앱

localhost:8501

```
$ streamlit run streamlit_app.py
```

가장 빠르게 기능을 확인할 수 있는 통합 대시보드 모드입니다.



React + FastAPI (분리형)

localhost:8000 / 5173

```
$ uvicorn backend.app.main:app --reload
```

```
$ cd frontend && npm i && npm run dev
```

실서비스와 유사한 프론트/백엔드 분리 환경입니다.



Tkinter 데스크톱

Native Window

```
$ python app.py
```

브라우저 없이 로컬 윈도우 창으로 실행됩니다.

상세 아키텍처 & 에러 처리

 news_client.py

Data Ingestion

NewsAPI 'everything' 엔드포인트 호출 및 원시 데이터 필터링

API 오류(429/401/5xx) 발생 시 상태 코드 그대로 전파 → Main에서 처리

 pipeline.py

Orchestrator

고유 ID 생성(SHA256), 데이터 정규화, 전체 흐름 제어

개별 기사 처리 실패 시 전체 중단 대신 'Empty/Error' 객체 반환

 summarizer / sentiment

AI Analysis

텍스트 요약 및 감정 분석 (3-Level Fallback)

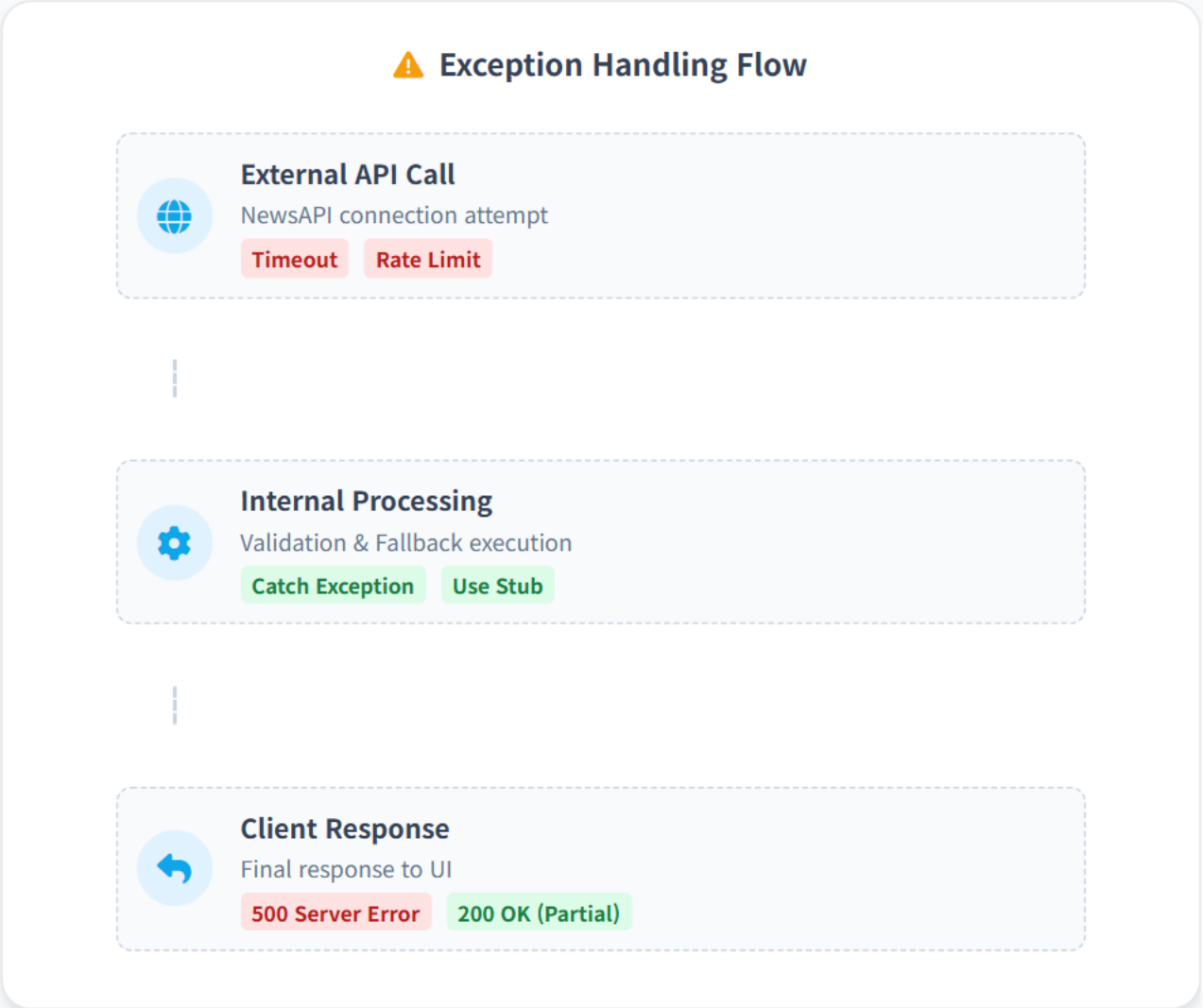
OpenAI Timeout/Auth Error → Transformers → Stub으로 자동 전환

 main.py

Backend API

FastAPI 엔드포인트 제공 (/api/fetch, /api/health)

최종적으로 클라이언트에게 표준 HTTP 에러(502 Bad Gateway 등) 반환



부록: 데이터 스키마 상세

Full JSON Structure & Field Dictionary

response_structure.json

```
{ "query": "AI Ethics", "language": "en", "fetched_at": "2026-01-04T14:30:00Z",  
  "articles": [ { "id": "f8a9 ... b2c", // SHA256(url) "title": "The Rise of  
Ethical AI", "url": "https://techcrunch.com/ ... ", "source": "TechCrunch",  
  "publishedAt": "2026-01-03T09:00:00Z",  
  "text": "Artificial intelligence is ... ", // 정규화된 본문  
  "summary": "Experts discuss guidelines ... ", // AI 생성 요약  
  "sentiment": { "label": "positive", "score": 0.92, "model": "openai-gpt-3.5-  
turbo" }  
}, { // Fallback Example "id": "e4d2 ... a1b", "title": "AI Risks Analysis",  
  "sentiment": { "label": "neutral", "score": 0.0, "model": "vader"  
  // 로컬 플백 모델 사용 } } ] }
```



기본 메타데이터 (Metadata)

검색 쿼리 정보(`query`, `language`)와 수집 시점
(`fetched\_at`)을 최상위에 명시하여, 데이터의 맥락(Context)
을 보존합니다. 기사 원본의 출처와 발행일도 표준 포맷으로 유지됩
니다.

Context Preservation

ISO 8601 Date



텍스트 & 요약 (Text & Summary)

`text`는 제목/요약/본문을 병합하고 공백을 정리한 정규화된 텍
스트입니다. `summary`는 설정된 모델
(OpenAI/Transformers/Stub)에 의해 생성된 요약문입니다.

text_utils.py

Normalization



감정 분석 객체 (Sentiment Object)

단순 점수뿐만 아니라 `model` 필드를 포함하여, 어떤 엔진
(OpenAI, VADER, Rule-based)이 사용되었는지 추적 가능합니
다. 이는 품질 모니터링에 필수적입니다.

Model Tracking

Observability

참고 자료 및 링크



라이브 데모



Streamlit 통합 앱

miniproject02.streamlit.app



React + FastAPI 분리형 앱

miniproject02.vercel.app



활용 기술 문서



NewsAPI Documentation

newsapi.org/docs



Hugging Face Transformers

huggingface.co/docs/transformers



VADER Sentiment Analysis

github.com/cjhutto/vaderSentiment



소스 코드 & 문서



GitHub Repository

github.com/username/miniproject02-main



README.md (설치 가이드)

/blob/main/README.md



배포 플랫폼



Streamlit Community Cloud

streamlit.io/cloud



Render (Backend Hosting)

render.com



Vercel (Frontend Hosting)

vercel.com

